CFG NINJA

SECURITY ASSESSMENT
SteelMineV2 Game

July 23, 2026

Audit Status: Pass

BLADE POOL

RISK ANALYSIS | SteelMineV2.

Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (https://www.first.org/cvss/v4.0/specification-document). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including honeypot detection, ownership structure, and trading restrictions.

Classification	CVSS Range	Description
Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
Informational	0.0	Informational findings about detected functions or contract features.

Manual Code Review Risk Results

Contract Security	Description
Buy Tax	N/A
Sale Tax	N/A
Cannot Buy	N/A
Cannot Sale	N/A
Max Tax	N/A
Modify Tax	N/A
Fee Check	Not Applicable
Is Honeypot?	Not Applicable
Trading Cooldown	Not Detected
Enable Trade?	N/A

Contract Security	Description
Pause Transfer?	Not Detected
Max Tx?	Not Detected
Is Anti Whale?	Not Detected
Is Anti Bot?	Not Detected
Is Blacklist?	Not Detected
Blacklist Check	Not Detected
Is Whitelist?	Not Detected
Can Mint?	Mints STEEL via minter role on VeinToken
Is Proxy?	Not Detected
Can Take Ownership?	Owner controls oracle/wiring/seed (see CFG20)
Hidden Owner?	Not Detected
Owner	Owner-controlled (OpenZeppelin Ownable)
Self Destruct?	Not Detected
External Call?	Present (trusted wiring)
Other?	Round lottery, drand oracle, refinable STEEL, motherlode jackpot, auto-subscribe keeper
Holders	N/A
Audit Confidence	Good - Pass; elevated centralization risk noted (owner-swappable oracle)
Authority Check	Owner-controlled
Freeze Check	N/A

This summary provides an overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.

CFG Ninja Verified on July 23, 2026

SteelMineV2

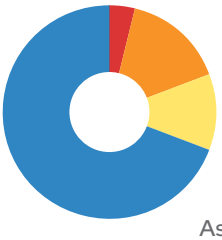
Executive Summary

TYPES	ECOSYSTEM	LANGUAGE
DeFi	Robinhood Chain	Solidity

Timeline

Audit Request	Onboarding Process	Audit Preview	Audit Release
2026-07-22	2026-07-22	2026-07-22	2026-07-22

Vulnerability Summary



26	18	8	8
Total Findings	Resolved	Pending	Unresolved

Assessed against the CFG01-CFG26 framework with Slither 0.11.5 static analysis.

Severity	Resolution Status	Description
0 Critical		Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise.
1 High	0 Resolved, 1 Pending	Significant risk that should be addressed urgently to prevent exploitation or financial loss.
4 Medium	0 Resolved, 4 Pending	Moderate risk that can lead to issues if combined with other weaknesses; should be addressed.
3 Low	0 Resolved, 3 Pending	Minor concern or best-practice deviation with limited direct security impact.
18 Informational	18 Resolved, 0 Pending	Not a vulnerability - code-quality, documentation, or optimization suggestions.

Industry Standards Compliance

This audit follows internationally recognized security standards to provide comprehensive assurance.

EEA EthTrust Security Levels

EthTrust/EVM review conducted for a Robinhood Chain contract.

Smart Contract Security Verification Standard (SCSVS)

SCSVS review conducted for an EVM smart contract on Robinhood Chain.

CVSS v3.1 Severity Scoring

Per-finding CVSS recorded on each CFG item.

PROJECT OVERVIEW | SteelMineV2.

Token Summary

Parameter	Result
Address	0xB089d11432F219495A4278acd6446B7Faefa2bA6
Name	SteelMineV2
Token Tracker	SteelMineV2 (STEEL)
Decimals	N/A
Supply	N/A (game contract)
Platform	Robinhood Chain
Compiler	v0.8.33+commit.64118f21
Contract Name	SteelMineV2
Optimization	Yes with 200 runs
LicenseType	MIT (SPDX)
Language	Solidity
Codebase	https://robinhoodchain.blockscout.com/address/0xB089d11432F219495A4278acd6446B7Faefa2bA6?tab=contract

Main Contract Assessed

Name	Contract	Live
SteelMineV2	0xB089d11432F219495A4278acd6446B7Faefa2bA6	Yes
VeinToken (STEEL)	0x0AF77e27F535256965944E617a386570f5C0432a	Yes
VeSteelV2 (Locking)	0xD5116ca699eD6CA186BC07f46B3c851D1A483aa2	Yes

TestNet Contract Was Not Assessed

Solidity Code Provided

SolID	File Sha-1	FileName
SteelMineV2	54cc8b98e43d842ca216ea81ad7b96b9d8d32929	SteelMineV2.sol

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Slither Static Analysis Results

Slither is a Solidity static analysis framework that runs a suite of vulnerability detectors to identify potential security issues, code quality problems, and best practice violations. It provides comprehensive security analysis with confidence ratings for each finding. For full detector documentation, visit: https://github.com/crytic/slither/wiki/Detector-Documentation

Impact Level	Count	Description
High	2	Critical security vulnerabilities requiring immediate attention (e.g., reentrancy, unchecked transfers)
Medium	4	Significant issues that should be addressed (e.g., dangerous comparisons, arithmetic issues)
Low	5	Minor issues and best practice violations (e.g., missing events, naming conventions)
Informational	59	Code quality and optimization suggestions (e.g., unused variables, gas optimizations)
TOTAL	**70**	**Total findings across all severity levels**

Analysis Summary

Slither 0.11.5 (solc 0.8.33, --via-ir) reported 70 results. Triaged material items: weak-prng and arbitrary-send on resolveRound (winner selection derives from the owner-swappable drand oracle and pays out - the core of CFG20); reentrancy-eth/-benign on resolveRound and reentrancy-no-eth on claimAll (guarded by nonReentrant, trusted callees - CFG19); divide-before-multiply and dangerous strict-equality in the emission/auto paths; costly-loop in seedUnrefined; and pervasive block.timestamp comparisons for round timing (CFG17). Informational: unused _claimedRound mapping, 'steel' parameter shadowing in seedMotherlode, zero-defaulted uninitialized locals, and OpenZeppelin dead-code - no security impact.

Remediation Approach

All High and Medium impact findings from Slither have been manually reviewed and mapped to CFG test cases where applicable. Each finding has been analyzed in the context of the contract's business logic to determine actual exploitability. Detailed analysis of specific findings is available in the Technical Findings section of this audit report.

Deprecation Notice

The Smart Contract Weakness Classification Registry (SWC Registry) was deprecated in September 2023. CFGNinja audits now use Slither for static analysis, which provides more comprehensive and up-to-date vulnerability detection. SWC was an implementation of the weakness classification scheme proposed in EIP-1470, loosely aligned to the Common Weakness Enumeration (CWE). For historical reference: https://swcregistry.io/

Detailed Security Findings

Each finding lists its severity, CVSS, location and description, the affected source excerpt (verbatim from the verified Blockscout source), our recommendation, mitigation, references, and a suggested fix where the remediation is code-expressible.

STEEL-04 | Unbounded Winner Loop Can Permanently Lock a Round..

Category	Severity	CVSS	Location	Status
Denial of Service	Medium	6.0	SteelMineV2.sol: SteelMineV2.sol : _settleWinners / _pickSolo (L512-L538)	Detected

Description

_settleWinners and _pickSolo iterate over _bettors[rid][winner], the full list of unique addresses that bet the winning square, inside resolveRound. Winnings are only credited during resolution - there is no alternative claim path. If a winning square accumulates enough unique bettors that the settlement loop exceeds the block gas limit, resolveRound can never succeed and that round's entire pot and STEEL emission are locked forever. With minCommit defaulting to 0.0001 native units per square, an attacker can cheaply inflate bettor arrays across many addresses to grief rounds on a low-fee L2.

Recommendation

Cap distinct bettors per square, or convert settlement to a pull-based per-winner claim (store poolEth/winnerPot at resolve and let each winner claim their pro-rata share) so a large winner set cannot brick resolution.

Mitigation

Gas-bounded settlement loop can strand a round's funds.

References:

https://swcregistry.io/docs/SWC-128

Affected Code

```
address[] storage bs = _bettors[rid][win];
uint256 len = bs.length;
for (uint256 i; i < len; i++) {              // unbounded -> gas DoS
    address u = bs[i];
    uint256 b = _bet[keccak256(abi.encode(rid, uint256(win), u))];
    ...                                       // credit each winner here
}
```

Suggested Fix

```
// pull-based: store poolEth/winnerPot at resolve; each winner claims:
function claimWin(uint256 rid) external nonReentrant {
    uint256 b = _bet[key(rid, rounds[rid].winner, msg.sender)];
    // credit msg.sender pro-rata, mark claimed  (O(1), no round-wide loop)
}
```

STEEL-05 | Missing Events on Admin Wiring..

Category	Severity	CVSS	Location	Status
Logging	Low	2.5	SteelMineV2.sol: SteelMineV2.sol : setVeSteel / setDrand / setDevAddr (L646-L657)	Detected

Description

setVeSteel, setDrand, setDevAddr and setAutoFee change trust-critical wiring but emit no dedicated events (only economics changes emit ParamsChanged), so oracle and veSTEEL swaps are not visible in event feeds - compounding CFG20.

Recommendation

Emit DrandUpdated, VeSteelUpdated and DevAddrUpdated events.

Mitigation

Trust-critical wiring changes are unlogged.

References:

https://consensys.github.io/smart-contract-best-practices/

Affected Code

```
function setVeSteel(IVeSteelV2 v) external onlyOwner { veSteel = v; }
function setDrand(IDrandOracle d) external onlyOwner { drand = d; }   // no events
function setDevAddr(address d) external onlyOwner { ... devAddr = d; }
```

Suggested Fix

```
event DrandUpdated(address oracle);
function setDrand(IDrandOracle d) external onlyOwner {
    drand = d; emit DrandUpdated(address(d));
}
```

STEEL-11 | Oracle Liveness Dependency - No Emergency Path..

Category	Severity	CVSS	Location	Status
Availability	Medium	5.5	SteelMineV2.sol: SteelMineV2.sol : resolveRound (L428-L430)	Detected

Description

If the drand beacon stalls or never finalizes the target round, drand.randomness(dr) returns bytes32(0) and resolveRound reverts with NotReady indefinitely. There is no timeout, no fallback resolution, and no admin/emergency function to refund or unwind an unresolvable round, so all funds committed to affected rounds are stranded with no recovery path.

Recommendation

Add a bounded liveness fallback: after a grace period with no beacon, allow the round to be voided and committed funds refunded pro-rata. Monitor beacon liveness off-chain.

Mitigation

No recovery path if the beacon fails to deliver randomness.

References:

https://owasp.org/www-project-smart-contract-top-10/

Affected Code

```
uint64 dr = drandRoundFor(rid);
bytes32 rand = drand.randomness(dr);
if (rand == bytes32(0)) revert NotReady();    // stalls forever if beacon dies
```

Suggested Fix

```
if (rand == bytes32(0)) {
    if (block.timestamp > bettingEndsAt(rid) + GRACE) _void(rid); // refund pot
    else revert NotReady();
}
```

STEEL-13 | Cap Accounting Edge Near miningCap..

Category	Severity	CVSS	Location	Status
Accounting	Low	3.0	SteelMineV2.sol: SteelMineV2.sol : _emit / resolveRound (L551-L562, L484)	Detected

Description

Motherlode STEEL is only reserved against miningCap (via emittedTotal) when a jackpot pays out, while _emit sizes new emissions from emittedTotal. Near the cap, a jackpot reserving a large accumulated motherlode plus concurrent refine mints can push the token past its hard cap, at which point the token's mint reverts with CapExceeded and the affected claim/resolution reverts. The hard cap is never breached, but claims can brick as the cap is approached.

Recommendation

Reserve motherlode STEEL against the cap at emission time, or clamp payouts to remaining mintable supply so claims cannot revert.

Mitigation

Late motherlode reservation can cause claim reverts near the cap.

References:

https://swcregistry.io/docs/SWC-101

Affected Code

```
emittedTotal += winnerSteel + devSteel;   // motherlode NOT reserved here
// ...only later, on a jackpot hit:
emittedTotal += motherlodeSteel;          // reserved late -> can exceed cap
```

Suggested Fix

```
// clamp payout to remaining cap so claims never revert:
uint256 room = miningCap - emittedTotal;
poolSteel = poolSteel < room ? poolSteel : room;   // or reserve at emit time
```

STEEL-16 | Owner Migration Seeding..

Category	Severity	CVSS	Location	Status
Centralization	Medium	5.5	SteelMineV2.sol: SteelMineV2.sol : seedUnrefined (L669-L678)	Detected

Description

Before finalizeSeed(), seedUnrefined(users, amounts) lets the owner credit arbitrary unrefined STEEL (reserved against the cap, so it mints on refine) to any addresses, and seedMotherlode sets the motherlode balances. The owner can allocate mintable STEEL to itself up to the cap accounting. This is a migration tool but concentrates significant trust in the owner key until the window is closed.

Recommendation

Call finalizeSeed() immediately after migration, publish the seeded balances, and hold ownership in a multisig.

Mitigation

Owner can seed arbitrary mintable STEEL/motherlode until finalizeSeed().

References:

https://swcregistry.io/docs/SWC-105
https://owasp.org/www-project-smart-contract-top-10/

Affected Code

```
function seedUnrefined(address[] calldata users, uint256[] calldata amounts)
    external onlyOwner {
    if (seeded) revert BadParam();
    for (uint256 i; i < users.length; i++) {
        _accrueUnrefined(users[i], amounts[i]);   // arbitrary mintable STEEL
        emittedTotal += amounts[i];
    }
}
```

Suggested Fix

```
// finalize immediately after migration; hold owner in a multisig:
function finalizeSeed() external onlyOwner {
    seeded = true; emit SeedFinalized();          // publish seeded balances
}
```

STEEL-19 | Checks-Effects-Interactions Ordering in resolveRound..

Category	Severity	CVSS	Location	Status
Security	Low	3.5	SteelMineV2.sol: SteelMineV2.sol : resolveRound / claimAll (L458-L500)	Detected

Description

resolveRound makes external calls (veSteel.notifyReward, steel.mint, veSteel.creditDevLock) and settles winner funds while some Round fields are written afterward (Slither: reentrancy-eth/-benign); claimAll mints STEEL before zeroing balances (reentrancy-no-eth). All are guarded by nonReentrant and the callees are trusted wiring, so they are not currently exploitable, but the ordering violates checks-effects-interactions.

Recommendation

Finalize all Round and miner state before external calls; keep the trusted-callee assumption documented.

Mitigation

External calls precede some state writes; mitigated by nonReentrant and trusted callees.

References:

https://swcregistry.io/docs/SWC-107

Affected Code

```
if (devSteel > 0) _mintDev(devSteel);                       // external
if (stakerEth > 0 && address(veSteel) != address(0))
    veSteel.notifyReward{value: stakerEth}();               // external
...
r.winnersPool = uint128(poolEth);                           // state AFTER calls
```

Suggested Fix

```
// finalize all Round/miner state BEFORE external calls, e.g.:
r.winnersPool = uint128(poolEth);
r.steelForWinners = uint128(poolSteel);
// ...then _mintDev / notifyReward / settle (callees remain trusted wiring).
```

STEEL-20 | Owner-Swappable Randomness Oracle..

Category	Severity	CVSS	Location	Status
Centralization / Randomness	High	8.2	SteelMineV2.sol: SteelMineV2.sol : setDrand / resolveRound (L650-L652, L435)	Detected

Description

The fairness of the entire game rests on drand.randomness(round), and setDrand(IDrandOracle) lets the owner replace the oracle at any time with no timelock and no event. Because the winning square is randomness % SQUARES, an owner who points drand at an oracle they control can effectively choose the winning square for future rounds and drain pots to addresses they control (Slither corroborates with weak-prng and arbitrary-send on resolveRound). The anti-look-ahead design (beacon finalizes after betting) only protects players if the oracle is honest and immutable; owner control of the oracle nullifies it. The wired oracle address could not be confirmed on-chain, and the IDrandOracle source itself warns the live ABI must be confirmed before use.

Recommendation

Make the drand oracle immutable, or place setDrand behind a timelock + multisig with an event, and publish the oracle's provenance. Prefer a verifiable VRF or commit-reveal so no single party can choose the winner.

Mitigation

Owner can swap the randomness oracle and thereby influence winner selection.

References:

https://swcregistry.io/docs/SWC-120
https://owasp.org/www-project-smart-contract-top-10/
https://swcregistry.io/docs/SWC-105

Affected Code

```
function setDrand(IDrandOracle d) external onlyOwner {
    drand = d;                    // owner can swap the RNG source anytime
}
// in resolveRound():
uint8 win = uint8(uint256(rand) % SQUARES);   // rand = drand.randomness(dr)
```

Suggested Fix

```
IDrandOracle public immutable drand;          // fix at deploy, no setter
// or gate setDrand behind timelock+multisig with an event, and publish
// the beacon's provenance; prefer a verifiable VRF / commit-reveal.
```

STEEL-23 | Anonymous Team - No KYC Verification..

Category	Severity	CVSS	Location	Status
Team Transparency	Medium	5.0	SteelMineV2.sol: Project team	Detected

Description

No team identities or KYC are disclosed. For a game contract that custodies user funds, mints the reward token and relies on an owner-controlled oracle, an anonymous team materially compounds the centralization findings above.

Recommendation

Complete KYC with a recognized provider and disclose at least a team-lead identity.

Mitigation

No KYC or team disclosure found.

References:

https://consensys.github.io/smart-contract-best-practices/

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
Critical	0	0	0
High	1	1	0
Medium	4	4	0
Low	3	3	0
Informational	18	0	18
Total	26	8	18

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | SteelMineV2.

Social Media	URL	Result
Website	https://www.steelrh.fun/	Pass
Telegram	https://discord.com/invite/DUVvVKSyr	Pass
Twitter	https://x.com/steeldotfun	Pass
Facebook		N/A
Reddit	N/A	N/A
Instagram	N/A	N/A
CoinGecko		N/A
Github	N/A	N/A
CMC		N/A
Email		Contact
Other	https://discord.com/invite/DUVvVKSyr	Pass

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project website (steelrh.fun), X/Twitter (@steeldotfun) and Discord confirmed. No CoinGecko/CMC listing at time of audit. Team is anonymous.

Project Owner Notes: Social channels confirmed active.

Assessment Results | Final Audit Score STEEL.

Review	Score
Security Score	80
Auditor Score	80

Our security assessment or audit score system for the smart contract and project follows a comprehensive evaluation process to ensure the highest level of security. The system assigns a score based on various security parameters and benchmarks, with a passing score set at 80 out of a total attainable score of 100.The assessment process includes a thorough review of the smart contracts codebase, architecture, and design principles. It examines potential vulnerabilities, such as code bugs, logical flaws, and potential attack vectors. The evaluation also considers the adherence to best practices and industry standards for secure coding. Additionally, the system assesses the projects overall security measures, including infrastructure security, data protection, and access controls. It evaluates the implementation of encryption, authentication mechanisms, and secure communication protocols. To achieve a passing score, the smart contract and project must attain a minimum of 80 points out of the total attainable score of 100. This ensures that the system has undergone a rigorous security assessment and meets the required standards for secure operation.

Important Notes for STEEL

SteelMineV2 (Mining Lottery) - Smart Contract Security Audit Report

PROJECT OVERVIEW

SteelMineV2 is the core game contract of the STEEL (steel.fun) protocol on Robinhood Chain - a round-based, grid-mining lottery inspired by slvr.fun. Each round, miners commit ETH to squares on a grid; a drand randomness beacon selects the winning square once betting closes. Winners split that round's ETH pot pro-rata and receive newly minted STEEL rewards. The contract layers in a motherlode jackpot (1/625 roll), a single-miner "mining jackpot" variance mode, an ORE-style refining mechanic (winnings are unrefined STEEL that costs 10% to mint, redistributed to other unrefined holders), an auto-subscription keeper, and fee streaming to veSTEEL lockers and the dev permanent lock. This report covers the SteelMineV2 contract only.

CONTRACT DETAILS

Name: SteelMineV2

Contract Address: 0xB089d11432F219495A4278acd6446B7Faefa2bA6

Chain: Robinhood Chain (EVM, Cancun)

Compiler: Solidity v0.8.33+commit.64118f21 | Optimization: Enabled, 200 runs | EVM: Cancun (requires via-IR)

Verification: Verified on Blockscout

Base Libraries: OpenZeppelin Ownable, ReentrancyGuard; custom IDrandOracle interface

Dependencies: VeinToken (STEEL mint), VeSteelV2 (ETH + dev-lock), IDrandOracle (randomness)

Explorer: https://robinhoodchain.blockscout.com/address/0xB089d11432F219495A4278acd6446B7Faefa2bA6?tab=contract

GAME MECHANICS

• Grid & rounds: SQUARES-square grid (25 in the reference design), fixed roundDuration (60s design). Betting closes `bettingDuration` into a round; the rest is the reveal phase.

• Randomness: winner = drand.randomness(round) % SQUARES. The target drand round is chosen so the beacon only finalizes AFTER betting closes, so bettors cannot see the outcome while betting. `resolveRound` is permissionless once betting closes and the beacon is available.

• ETH split (from a 10% protocol fee of the pot): 90% to winners (always pro-rata by bet), remainder of the fee to veSTEEL stakers, 2% to the motherlode, 2% keeper/dev. Winners' ETH is ALWAYS pro-rata - never winner-take-all.

• STEEL emission (reserved 1.12x against miningCap): 1.0 to winners, 0.08 to the dev permanent lock, 0.04 to the motherlode. On a solo round the whole winner emission goes to one bet-weighted winner (the mining jackpot).

• Motherlode: ETH + STEEL that pays out to a round's winners on a 1/625 roll.

• Refining: winnings accrue as `unrefined` STEEL; claimAll mints it minus a 10% refining fee, which is redistributed pro-rata to other unrefined holders via a dividend index.

• Auto-subscribe: users escrow ETH and a keeper (permissionless autoExecuteMany) commits their squares each round; reload mode compounds winnings back into the escrow.

• Admin: owner can set the drand oracle, veSTEEL, devAddr, auto-fee, betting window, min commit, and economics (bounded), plus one-shot migration seeding.

SECURITY CONCERNS

Owner-Swappable Randomness Oracle (HIGH - CFG20)

The entire fairness of the game rests on `drand.randomness(round)`, and `setDrand(IDrandOracle)` lets the owner replace the oracle at any time with no timelock and no event. A malicious or compromised owner can point `drand` at an oracle they control and, because the winning square is `randomness % SQUARES`, effectively choose the winning square for future rounds - draining pots to addresses they control (Slither corroborates with weak-prng and arbitrary-send-eth on `resolveRound`). The anti-look-ahead design (beacon finalizes after betting) only protects players if the oracle is honest and immutable; owner control of the oracle nullifies it. We could not confirm on-chain what oracle address is currently wired or whether it is a trusted drand relay.

Recommendation: Make the drand oracle immutable, or place `setDrand` behind a timelock + multisig and emit an event on change. Publish the oracle address and its provenance (the canonical Robinhood Chain

drand beacon). Consider a commit-reveal or VRF with on-chain verifiability so the winner cannot be chosen by any single party.

Unbounded Winner Loop Can Permanently Lock a Round (MEDIUM - CFG04)

`_settleWinners` (and `_pickSolo`) iterate over `_bettors[rid][winner]`, the full list of unique addresses that bet on the winning square, inside `resolveRound`. Winnings are only ever credited during resolution - there is no alternative per-user claim path. If a winning square accumulates enough unique bettors that the settlement loop exceeds the block gas limit, `resolveRound` can never succeed and that round's entire ETH pot and STEEL emission are locked forever. Because `minCommit` defaults to 0.0001 ETH per square, an attacker can cheaply inflate bettor arrays across many addresses to grief a round (they cannot predict the winning square, so bloating every square is required, but it remains economically feasible on a low-fee L2).

Recommendation: Cap the number of distinct bettors per square, or replace the settlement loop with a pull-based per-winner claim (store the round's poolEth / winnerPot and let each winner claim their pro-rata share in their own transaction) so a large winner set cannot brick resolution.

Oracle Liveness Dependency - No Emergency Path (MEDIUM - CFG11)

If the drand beacon stalls or never finalizes the target round, `drand.randomness(dr)` returns bytes32(0) and `resolveRound` reverts with `NotReady` indefinitely. There is no timeout, no fallback resolution, and no admin/emergency function to refund or unwind an unresolvable round. All ETH committed to affected rounds is stranded with no recovery path.

Recommendation: Add a bounded liveness fallback: after a grace period with no beacon, allow the round to be voided and committed ETH refunded pro-rata (or resolved from an alternative agreed source). Monitor beacon liveness off-chain.

Owner Migration Seeding (MEDIUM - CFG16)

Before `finalizeSeed()`, `seedUnrefined(users, amounts)` lets the owner credit arbitrary unrefined STEEL (reserved against the cap, so it mints on refine) to any addresses, and `seedMotherlode` sets the motherlode balances. The owner can therefore allocate mintable STEEL to itself up to the cap accounting. This is a migration tool but concentrates significant trust in the owner key until the window is closed.

Recommendation: Call `finalizeSeed()` immediately after migration, publish the seeded balances, and hold ownership in a multisig.

Checks-Effects-Interactions Ordering in resolveRound (LOW - CFG19)

`resolveRound` makes external calls (`veSteel.notifyReward{value}`, `steel.mint`, `veSteel.creditDevLock`) and settles winner ETH while some `Round` fields are written afterward (Slither: reentrancy-eth / reentrancy-benign on `rounds`, `emittedTotal`); `claimAll` mints STEEL before zeroing balances (reentrancy-no-eth). All are guarded by `nonReentrant` and the callees (steel, veSteel) are trusted wiring, so they are not currently exploitable, but the ordering violates checks-effects-interactions.

Recommendation: Finalize all `Round` and miner state before external calls; keep the trusted-callee assumption documented.

Cap Accounting Edge Near miningCap (LOW - CFG13)

Motherlode STEEL is only reserved against `miningCap` (via `emittedTotal`) when a jackpot actually pays out, while `_emit` sizes new emissions from `emittedTotal`. Near the cap, a jackpot that reserves a large accumulated motherlode plus concurrent refine mints can push the token past its hard cap, at which point `VeinToken.mint` reverts with CapExceeded and the affected `claimAll` / resolution reverts. The hard cap is never breached (the token enforces it), but claims can brick as the cap is approached.

Recommendation: Reserve motherlode STEEL against the cap at emission time (not at payout), or clamp payouts to remaining mintable supply so claims cannot revert.

Missing Events on Admin Wiring (LOW - CFG05)

`setVeSteel`, `setDrand`, `setDevAddr` and `setAutoFee` change trust-critical wiring but emit no dedicated events (only `setEconomics` / a few emit `ParamsChanged`). Oracle and veSTEEL swaps are therefore not visible in event feeds - compounding CFG20.

Recommendation: Emit `DrandUpdated`, `VeSteelUpdated`, `DevAddrUpdated` events.

Anonymous Team - No KYC (MEDIUM - CFG23)

No team member names, credentials, or verifiable identities are disclosed, and no KYC has been completed with a recognized provider. For a game contract that custodies user funds, mints the reward token, and relies on an owner-controlled oracle, an anonymous team materially compounds the centralization findings above (CFG20, CFG16).

Recommendation: Complete KYC with a recognized provider (Assure DeFi, PinkKYC, Solidproof) and disclose at least a team-lead identity.

INFORMATIONAL OBSERVATIONS

Block Timestamp Round Boundaries (INFORMATIONAL - CFG17)

Round and betting boundaries are computed from `block.timestamp` throughout. Validator timestamp nudging (a few seconds) is negligible relative to the round duration and cannot move the winning square (which comes from the beacon), so the impact is limited to edge-of-window commit timing. Reclassified informational - no action required at the current round durations.

POSITIVE FINDINGS

ReentrancyGuard: All external state-changing entrypoints (commit, resolveRound, claimEth, claimAll, auto* functions) are nonReentrant.

ETH Always Pro-Rata: Winner ETH is split by bet size and never winner-take-all, so solo/variance modes cannot let one player extract another's ETH - only the STEEL emission is subject to the lottery.

Anti-Look-Ahead Randomness Design: The target drand round is chosen to finalize after betting closes, so (with an honest oracle) bettors cannot see the outcome while committing.

```
Bounded Economic Parameters: `setEconomics` enforces caps (protocol fee <= 20%, jackpot +keeper <= protocol fee, steelPerRound <= 1000e18, dev/motherlode <= 20%, refining <= 20%); `setAutoFee` <= 0.01 ETH; `setMinCommit` <= 1 ETH.
```

Permissionless Resolution & Keeper: Anyone can call `resolveRound` and `autoExecuteMany`, so liveness does not depend on a privileged keeper (only on the beacon).

```
Cap Reservation: Winner + dev STEEL is reserved against `miningCap` at emission time; the token's own immutable cap is the hard backstop.
```

Solidity 0.8.33: Native overflow/underflow protection.

Static Analysis: Slither 0.11.5 (solc 0.8.33, via-IR) produced 70 raw results. After triage the material items are the owner-controlled PRNG/oracle, the arbitrary-send (by-design winner payout), the CEI/reentrancy-ordering notes (guarded), the costly seed loop, and timestamp usage - all captured above. Informational items (unused `_claimedRound`, `steel` param shadow in seedMotherlode, uninitialized-but-zero-defaulted locals, OZ dead-code) carry no security impact.

Contract Verified: Source published and verified on Blockscout.

AUDIT METHODOLOGY

```
Manual line-by-line review of the verified Solidity source (main contract plus OpenZeppelin and IDrandOracle additional sources, downloaded from the Robinhood Chain Blockscout API), CFG01-CFG26 framework mapping, and automated static analysis with Slither 0.11.5 compiled against solc 0.8.33 (matching the on-chain compiler) via solc-select, with OpenZeppelin remappings and `--via-ir` (the contract exceeds the stack limit under the legacy pipeline). Aderyn was not run (Rust/cargo toolchain not installed on this host); Slither provided full static-analysis coverage.
```

```
On-chain state (the wired drand oracle address and its trustworthiness, whether finalizeSeed has been called, current round, pot sizes, emittedTotal vs miningCap) could not be independently confirmed because no archive RPC / indexer for Robinhood Chain was available during this review. Findings are based on the verified source code; on-chain-dependent statements are explicitly marked unconfirmed. The IDrandOracle source itself notes the live oracle ABI "must be confirmed before pointing the mine at the live oracle" - reinforcing CFG20.
```

Audit Date: July 22, 2026 | Chain: Robinhood Chain (EVM) | Auditor: CFG.NINJA

APPENDIX A: AFFECTED CODE AND SUGGESTED FIXES

The following excerpts are taken verbatim from the verified Blockscout source and are also rendered as monospace boxes in the Detailed Security Findings section of the PDF.

CFG04 - Unbounded Winner Loop Can Permanently Lock a Round (Medium)

Location: SteelMineV2.sol : _settleWinners / _pickSolo (L512-L538)

Affected Code:

```
address[] storage bs = _bettors[rid][win];
uint256 len = bs.length;
for (uint256 i; i < len; i++) {              // unbounded -> gas DoS
    address u = bs[i];
    uint256 b = _bet[keccak256(abi.encode(rid, uint256(win), u))];
    ...                                       // credit each winner here
}
```

Suggested Fix:

// pull-based: store poolEth/winnerPot at resolve; each winner claims:

```
function claimWin(uint256 rid) external nonReentrant {
    uint256 b = _bet[key(rid, rounds[rid].winner, msg.sender)];
    // credit msg.sender pro-rata, mark claimed  (O(1), no round-wide loop)
}
```

CFG05 - Missing Events on Admin Wiring (Low)

Location: SteelMineV2.sol : setVeSteel / setDrand / setDevAddr (L646-L657)

Affected Code:

```
function setVeSteel(IVeSteelV2 v) external onlyOwner { veSteel = v; }
function setDrand(IDrandOracle d) external onlyOwner { drand = d; }   // no events
function setDevAddr(address d) external onlyOwner { ... devAddr = d; }
```

Suggested Fix:

```
event DrandUpdated(address oracle);
function setDrand(IDrandOracle d) external onlyOwner {
    drand = d; emit DrandUpdated(address(d));
}
```

CFG11 - Oracle Liveness Dependency - No Emergency Path (Medium)

Location: SteelMineV2.sol : resolveRound (L428-L430)

Affected Code:

```
uint64 dr = drandRoundFor(rid);
bytes32 rand = drand.randomness(dr);
if (rand == bytes32(0)) revert NotReady();    // stalls forever if beacon dies
```

Suggested Fix:

```
if (rand == bytes32(0)) {
    if (block.timestamp > bettingEndsAt(rid) + GRACE) _void(rid); // refund pot
    else revert NotReady();
}
```

CFG13 - Cap Accounting Edge Near miningCap (Low)

Location: SteelMineV2.sol : _emit / resolveRound (L551-L562, L484)

Affected Code:

```
emittedTotal += winnerSteel + devSteel;   // motherlode NOT reserved here
```

// ...only later, on a jackpot hit:

```
emittedTotal += motherlodeSteel;          // reserved late -> can exceed cap
```

Suggested Fix:

// clamp payout to remaining cap so claims never revert:

```
uint256 room = miningCap - emittedTotal;
poolSteel = poolSteel < room ? poolSteel : room;   // or reserve at emit time
```

CFG16 - Owner Migration Seeding (Medium)

Location: SteelMineV2.sol : seedUnrefined (L669-L678)

Affected Code:

```
function seedUnrefined(address[] calldata users, uint256[] calldata amounts)
    external onlyOwner {
    if (seeded) revert BadParam();
    for (uint256 i; i < users.length; i++) {
        _accrueUnrefined(users[i], amounts[i]);   // arbitrary mintable STEEL
```

```
        emittedTotal += amounts[i];
    }
}
```

Suggested Fix:

// finalize immediately after migration; hold owner in a multisig:

```
function finalizeSeed() external onlyOwner {
    seeded = true; emit SeedFinalized();          // publish seeded balances
}
```

CFG17 - Block Timestamp Round Boundaries (Informational)

Location: SteelMineV2.sol : currentRound / bettingEndsAt (L148-L164)

Affected Code:

```
function currentRound() public view returns (uint256) {
    if (block.timestamp < genesis) return 0;
    return (block.timestamp - genesis) / roundDuration;   // timestamp-based
}
```

Suggested Fix:

```
// Negligible at 60s+ rounds (validator skew << round; cannot move the
// beacon-derived winner). No change required at current durations.
```

CFG19 - Checks-Effects-Interactions Ordering in resolveRound (Low)

Location: SteelMineV2.sol : resolveRound / claimAll (L458-L500)

Affected Code:

```
if (devSteel > 0) _mintDev(devSteel);                        // external
if (stakerEth > 0 && address(veSteel) != address(0))
    veSteel.notifyReward{value: stakerEth}();                // external
...
r.winnersPool = uint128(poolEth);                            // state AFTER calls
```

Suggested Fix:

// finalize all Round/miner state BEFORE external calls, e.g.:

```
r.winnersPool = uint128(poolEth);
r.steelForWinners = uint128(poolSteel);
// ...then _mintDev / notifyReward / settle (callees remain trusted wiring).
```

CFG20 - Owner-Swappable Randomness Oracle (High)

Location: SteelMineV2.sol : setDrand / resolveRound (L650-L652, L435)

Affected Code:

```
function setDrand(IDrandOracle d) external onlyOwner {
    drand = d;                    // owner can swap the RNG source anytime
}
```

// in resolveRound():

```
uint8 win = uint8(uint256(rand) % SQUARES);   // rand = drand.randomness(dr)
```

Suggested Fix:

```
IDrandOracle public immutable drand;          // fix at deploy, no setter
// or gate setDrand behind timelock+multisig with an event, and publish
// the beacon's provenance; prefer a verifiable VRF / commit-reveal.
```

DISCLAIMER

This audit does not guarantee the absence of all vulnerabilities. This report covers the SteelMineV2 contract only; the VeinToken (STEEL) token and the VeSteelV2 locking contract are covered in separate reports. The game's fairness depends on the external drand oracle, which is outside the scope of this contract-level review and is itself the subject of the highest-severity finding. Users should conduct their own research before interacting with any smart contract.

Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how block.timestamp works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invokable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by Bladepool / CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.